

Semestrálny projekt - Nespojité cestná sieť

Tomáš Porubský (POR0087)

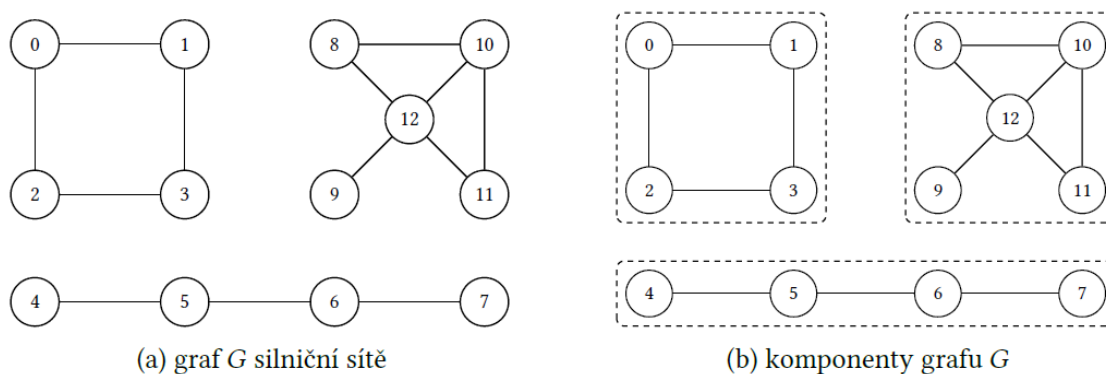
16. mája 2026

1 Zadanie projektu - problém

Zadané sú vrcholy neorientovaného grafu $G = (V, E)$, kde V je množina vrcholov (vertices) a E je množina hrán (edges). Vrcholy a hrany tvoria sieť, ktorú interpretujeme ako cestnú sieť v mestských častiach, pričom zadaná cestná sieť môže byť:

- súvislá a tvorí tak jeden celok - v tomto prípade žiadna oprava potrebná nie je
- nesúvislá - t.j. vrcholy a hrany tvoria vzájomne neprepojené celky.

V druhom prípade je potrebné vykonať opravu, ktorá pozostáva z odstránenia už existujúcich prepojení a pridania nových hrán/ciest medzi vrcholmi susediacich komponent / mestských častí. Ukážka zadania cestnej siete je vyobrazená na obrázku 1. Pri oprave je možné vytvoriť také množstvo nových ciest,

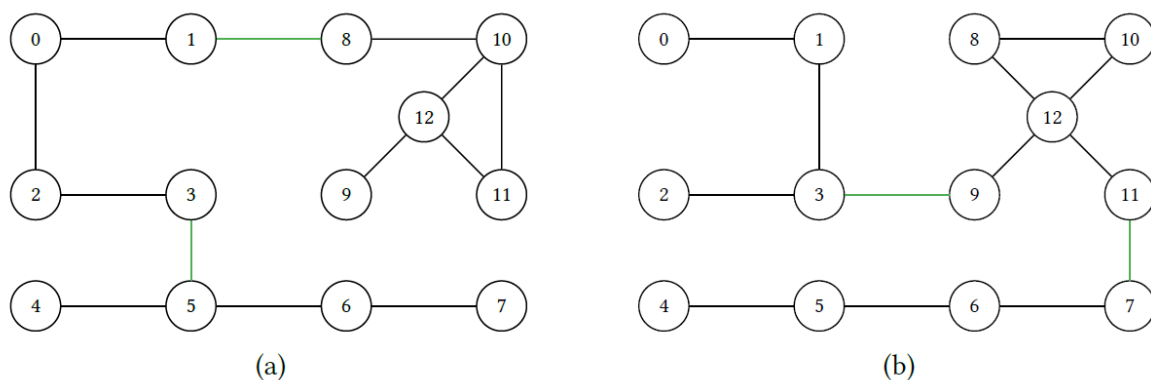


Obrázok 1: Chybne navrhnutá cestná sieť určená k oprave

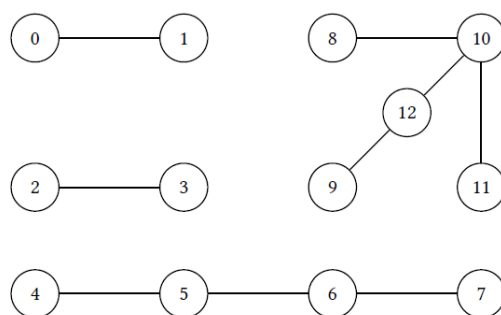
koľko bolo odobraných z pôvodného rozloženia. Kvôli požiadavku prepojenia komponent najmenším možným počtom vytvorených hrán je počet možností opravy limitovaný len na riešenia uberajúce $c - 1$ hrán, kde c označuje počet samostatných komponent. Dve možné riešenia opravy ukážkovej cestnej siete sú uvedené na obrázku 2. Nové prepojenia sú znázornené zelenou farbou.

Jeden z prípadov nesúvislej cestnej siete, ktorý môže nastať je neopraviteľná cestná sieť, zobrazená na obrázku 3. Zadaná sieť obsahuje totiž viac komponent než je počet nemostových ciest použiteľných k oprave. Pojem mostovej hrany / cesty definujeme ako hranu, pri ktorej odstránení dojde ku zvýšeniu počtu komponent.

Vrcholy a hrany grafu sú zadané v textovom súbore - každá hrana je na novom riadku, pričom vrcholy sú v riadku reprezentované dvojicou celých čísel i a j oddelených bielym znakom.



Obrázok 2: Správne opravená cestá sieť



Obrázok 3: Neopraviteľná cestná sieť

2 Návrh riešenia problému

Program pracuje s neorientovaným grafom reprezentovaným zoznamom hrán. Cieľom je analyzovať štruktúru grafu, zistiť počet jeho súvislých komponentov, identifikovať mosty a následne navrhnúť úpravu grafu tak, aby bol výsledný graf súvislý pri minimálnych zásahoch. Základným algoritmom, ktorý bol zvolený pre analýzu grafu je **Deep-first search**, ktorého funkciu je možné popísať takto: Algoritmus postupuje do maximálnej novej hĺbky grafu, kým nenarazí na vrchol, ktorého všetci susedia už boli navštívení (preskúmaní). Potom sa vracia späť do miesta rozvetvenia a pokračuje v prehľadávaní ďalších vetiev.

Stručný koncept riešenia je nasledovný:

1. **Načítanie vstupu**
2. **Tvorba zoznamu susedov - tzv. adjacency list**
3. **Prechod grafom pomocou DFS a určenie počtu komponentov**
4. **Hľadanie mostov**
5. **Návrh opravy grafu a kontrola výsledku**

Detailný popis riešenia problému je diskutovaný v nasledujúcich odsekoch.

2.1 Načítanie vstupu

Zo súboru sú načítané všetky hrany, pričom každá dvojica čísel predstavuje jednu neorientovanú hranu grafu. Zvolený dátový typ pre zoznam hrán je `vector<pair<int, int>>`.

2.2 Vytvorenie adjacency list

Pre prechádzanie grafom pomocou DFS v ďalšej fáze programu som z načítaného zoznamu hrán vytvoril **adjacency list** (zoznam bodov susediacich s daným bodom), ktorý má štruktúru `vector<vector<int>>` a vyzerá napríklad takto:

```
0: 1 3
1: 0 2
2: 1 3
3: 2 0
4: 5 7
5: 4 6
6: 5 7
7: 6 4
```

2.3 Prechod grafom pomocou DFS

Algoritmus Depth-First Search (DFS) sa používa na prechod grafu, zisťovanie dosiahnuteľnosti vrcholov - celistvosti cestnej siete. DFS som využil pri zisťovaní počtu komponentov a pri ich označovaní.

Listing 1: Ukážka časti kódu

```
void dfsRec(vector<vector<int>> &adj, vector<bool> &visited, int s, vector<int> &res) {
    visited[s] = true;
    res.push_back(s);
    // Rekurzívne navštíví vrcholy, ktoré ešte neboli navštívene
    for (int i : adj[s])
        if (visited[i] == false)
            dfsRec(adj, visited, i, res);
}

vector<int> dfs(vector<vector<int>> &adj) {
    vector<bool> visited(adj.size(), false);
    vector<int> res;
    // Prepojenie nesúvislých častí grafu
    for (int i = 0; i < adj.size(); i++) {
        if (visited[i] == false) {
            dfsRec(adj, visited, i, res);
        }
    }
    return res;
}
```

V tejto fáze program zistí, koľko súvislých komponentov graf obsahuje. Ak je graf súvislý, nie je potrebná žiadna oprava a program sa ukončí s hláškou "Graf je spojený, není potreba zadné opravy." . V prípade, že graf obsahuje viac než jednu komponentu, program pokračuje ďalej.

2.4 Hľadanie mostov

Keď vieme, že pre dosiahnutie celistvého cestného systému bude potrebná rekonštrukcia, potrebujeme zistiť, ktoré cesty je možné zbúrať a namiesto nich postaviť nové. Búrať chceme nemostové cesty a teda tie ktorých odstránením nespôsobia osamostatnenie nejakého vrcholu alebo skupiny vrcholov. Zvolil som naivný algoritmus, ktorý má pomerne veľkú časovú náročnosť (odhadom $O(N^2)$), je ale veľmi ľahko pochopiteľný a naprogramovateľný. Spočíva v prechádzaní zoznamu hrán - *edges list*, v ktorom hľadá most pomocou volania funkcie `isBridge()`. Táto funkcia berie tri argumenty `edgeIndex`, `edgeList` a `removedList` a funguje tak, zostrojí nový graf - *adjacency list* ale už bez hrán, ktoré boli označené a uložené v `removedList`. Tým sa zaručí, že neodstránime hrany, ktoré sa mohli stať mostovými po odstránení predošlých hrán.

Algorithm 1: Algoritmus hľadania mostov

Funkcia `isBridge(edgeIndex, edges, removed, comp_count_original)`

// Zistíme najväčší vrchol

`maxv` ← maximum vrchol zo všetkých hrán v `edges`

// Vytvoríme prázdny graf s vrcholmi 0..`maxv`

`G` ← prázdny zoznam susedov veľkosti (`maxv` + 1)

// Pridáme všetky hrany okrem testovanej a odstránených

foreach hrana `i` v `edges` **do**

if `i` = `edgeIndex` **then**

 ↳ `continue`

if `i` ∈ `removed` **then**

 ↳ `continue`

 pridaj hranu `edges[i]` do `G`

// Spočítame počet komponentov po odstránení hrany

`after` ← `components_count(G)`

// Hrana je most, ak odstránenie zvýši počet komponentov

return `after` > `comp_count_original`

Podľa vykonanej rešerše dostupných algoritmov pre určenie mostových hrán by bol z pohľadu časovej náročnosti vhodnejší **Tarjanov algoritmus**, ktorý má časovú náročnosť $O(N)$. Môj návrh má asi $O(N^2)$.

2.5 Odstránenie nemostových hrán a prepojenie komponent

V predošlom kroku som našiel a označil hrany, ktoré je možné odstrániť. Pokiaľ je počet takýchto hrán menší maximálne o jednotku oproti počtu komponent, nie je možná oprava cestnej siete a program sa ukončí hláškou: "všetky hrany su mostami alebo nie je možné odstrániť dostatok ciest --> neni mozna oprava !". V opačnom prípade, že počet odstrániteľných hrán je väčší než *poet_komponent* - 1, program vypíše "Je mozna oprava...". Následne pre prepojenia nesúvislých častí grafu (komponent) je potrebné vytvoriť zoznam `vector<int>comp[v]`, v ktorom sú priradené čísla komponent jednotlivým vrcholom `v`. Z každej dvojice komponent sa potom vyberie dvojica reprezentantov, medzi ktorými sa vytvoria nové hrany. Vytvorenie hrán zabezpečí, že výsledný graf bude súvislý. Po vykonaní navrhnutých úprav sa opätovne spočíta počet komponentov, aby sa overilo, že graf je skutočne súvislý.

3 Časová zložitosť algoritmu

Časová náročnosť jednotlivých častí programu bola spočítaná pomocou AI a je nasledovná:

- **components_count(adj)** Prechod grafu pomocou DFS.

$$T(n) = O(V + E)$$

- **dfsRec, dfs, dfs_comp** Rekurzívny alebo iteratívny DFS.

$$T(n) = O(V + E)$$

- **make_adj_list(edges)** Nájde maximálny vrchol a vytvorí zoznam susedov.

$$T(n) = O(V + E)$$

- **find_bridges(edges)** Brute-force testovanie každej hrany: pre každú hranu sa vytvorí nový graf a spočíta počet komponentov.

$$T(n) = O(E \cdot (V + E)) = O(EV + E^2)$$

- **isBridge(edgeIndex, edges, removed)** Vytvorenie grafu bez vybraných hrán a výpočet počtu komponentov.

$$T(n) = O(V + E)$$

- **find_edges_to_remove(edges, num_components)** Pre každú hranu volá isBridge, kým neodstráni dostatočný počet hrán.

$$T(n) = O(E \cdot (V + E)) = O(EV + E^2)$$

- **edges_to_create(adj)** Výpočet komponentov a hľadanie reprezentantov.

$$T(n) = O(V + E)$$

Ako je vidieť, časovo najnáročnejším procesom je funkcia `find_edges_to_remove(edges, num_components)`.

4 Záver

Projekt ukázal praktické využitie základných grafových algoritmov pri analýze a oprave nesúvislej cestnej siete. Pomocou DFS bol určený počet komponentov aj ich vnútorná štruktúra a jednoduchým testovaním odstránenia hrán boli identifikované mosty, ktoré nie je možné z grafu odobrať. Na základe týchto informácií program navrhol minimálnu opravu siete odstránením vhodných nemostových hrán a vytvorením nových prepojení medzi komponentmi. Výsledkom je algoritmus, ktorý dokáže z nesúvislého grafu vytvoriť súvislú štruktúru pri zachovaní jednoduchosti implementácie.